FRANZ JOSEPH RYLE DEL VALLE

CAPSTONE PROJECT FOR AI-ML POST GRADUATE DIPLOMA OF ASIAN INSTITUTE OF MANAGEMENT

1. Introduction

1.1 Background

In the banking industry, understanding customer behavior is essential for delivering personalized services, improving customer retention, and optimizing marketing strategies. Traditional approaches often treat customers as a homogeneous group, leading to inefficient targeting and missed opportunities. By segmenting customers, banks can tailor their offerings to meet specific needs and preferences, ultimately leading to higher customer satisfaction and profitability.

1.2 Problem Statement

This project aims to segment bank customers into distinct groups based on their financial behavior and characteristics using unsupervised machine learning techniques, specifically focusing on the `BankChurners.csv` dataset.

1.3 Objectives

- Identify meaningful customer segments within the bank's customer base.
- Apply clustering algorithms to group similar customers.
- Provide actionable business recommendations based on the identified segments.

1.4 Scope and Limitations

- This analysis uses a publicly available dataset (`BankChurners.csv`).
- The project focuses on the application of clustering techniques for segmentation.

- No real-time deployment is included in this scope.

```
# FRANZ JOSEPH RYLE DEL VALLE
# 1. IMPORT LIBRARIES
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.metrics import silhouette_score

# 2. LOAD DATA
# Load the BankChurners.csv dataset
df = pd.read_csv("BankChurners.csv")
```

2. Problem Framing

2.1 Machine Learning Task

- **Type**: Unsupervised Learning (Clustering)

2.2 Success Metrics

- **Silhouette Score**: Measures how similar an object is to its own cluster compared to other clusters.
- **Davies-Bouldin Index**: Evaluates the ratio of within-cluster scatter to between-cluster separation.

2.3 Business KPIs

- Improved targeting efficiency for marketing campaigns.
- Increased product adoption rates.
- Better customer retention strategies.

3. Dataset Overview

3.1 Dataset Description

- **Source**: BankChurners.csv (publicly available dataset)
- This dataset contains information about bank customers, including their demographic details, credit card usage, and churn status. It's often used for predicting customer churn, but here we will leverage its features for segmentation.

3.2 Data Dictionary (Initial look - detailed dictionary will be generated based on columns)

Let's inspect the dataset to understand its structure, features, and initial observations.

4. Data Preprocessing & Feature Engineering

4.1 Data Cleaning

We will start by handling irrelevant columns and checking for missing values.

```
# Drop irrelevant columns
irrelevant_cols = ['CLIENTNUM',
                   'Attrition_Flag',
                   'Naive_Bayes_Classifier_Attrition_Flag_Card_Catego
                   'Naive_Bayes_Classifier_Attrition_Flag_Card_Catego
df_processed = df.drop(columns=irrelevant_cols)

# Check for missing values
print("\nMissing values before handling:\n", df_processed.isnull().su

# For simplicity, we'll fill any potential missing numerical values w
# Although info() suggests no missing values, this is a good general
for col in df_processed.columns:
    if df_processed[col].dtype == 'object': # Categorical columns
        df_processed[col] = df_processed[col].fillna(df_processed[col
    else: # Numerical columns
        df_processed[col] = df_processed[col].fillna(df_processed[col

print("\nMissing values after handling:\n", df_processed.isnull().sum
```

```
Missing values before handling:
 Customer_Age                0
Gender                      0
Dependent_count             0
Education_Level             0
Marital_Status              0
Income_Category             0
Card_Category               0
Months_on_book              0
Total_Relationship_Count    0
Months_Inactive_12_mon      0
Contacts_Count_12_mon       0
Credit_Limit                0
Total_Revolving_Bal         0
Avg_Open_To_Buy             0
Total_Amt_Chng_Q4_Q1        0
Total_Trans_Amt             0
Total_Trans_Ct              0
Total_Ct_Chng_Q4_Q1         0
Avg_Utilization_Ratio       0
dtype: int64

Missing values after handling:
 0
```

4.2 Encoding Categorical Features

We need to convert categorical features into numerical format for clustering algorithms. One-Hot Encoding will be used.

```
# Identify categorical columns
categorical_cols = df_processed.select_dtypes(include='object').colum

# Apply One-Hot Encoding
df_encoded = pd.get_dummies(df_processed, columns=categorical_cols, d

print("\nShape of DataFrame after One-Hot Encoding:", df_encoded.shap
print("\nFirst 5 rows of encoded DataFrame:\n", df_encoded.head())
```

```
Shape of DataFrame after One-Hot Encoding: (10127, 32)

First 5 rows of encoded DataFrame:
    Customer_Age  Dependent_count  Months_on_book  Total_Relationship_
0            45                3              39
1            49                5              44
2            51                3              36
3            40                4              34
4            40                3              21

   Months_Inactive_12_mon  Contacts_Count_12_mon  Credit_Limit  \
0                       1                      3       12691.0
1                       1                      2        8256.0
2                       1                      0        3418.0
3                       4                      1        3313.0
4                       1                      0        4716.0

   Total_Revolving_Bal  Avg_Open_To_Buy  Total_Amt_Chng_Q4_Q1  ...  \
0                  777          11914.0                 1.335  ...
1                  864           7392.0                 1.541  ...
2                    0           3418.0                 2.594  ...
3                 2517            796.0                 1.405  ...
4                    0           4716.0                 2.175  ...

   Marital_Status_Single  Marital_Status_Unknown  Income_Category_$40K
0                  False                   False
1                   True                   False
2                  False                   False
3                  False                    True
4                  False                   False
```

```
   Income_Category_$60K - $80K  Income_Category_$80K - $120K  \
0                         True                         False
1                        False                         False
2                        False                          True
3                        False                         False
4                         True                         False

   Income_Category_Less than $40K  Income_Category_Unknown  \
0                           False                    False
1                            True                    False
2                           False                    False
3                            True                    False
4                           False                    False

   Card_Category_Gold  Card_Category_Platinum  Card_Category_Silver
0               False                   False                 False
1               False                   False                 False
2               False                   False                 False
3               False                   False                 False
4               False                   False                 False

[5 rows x 32 columns]
```

4.3 Feature Scaling

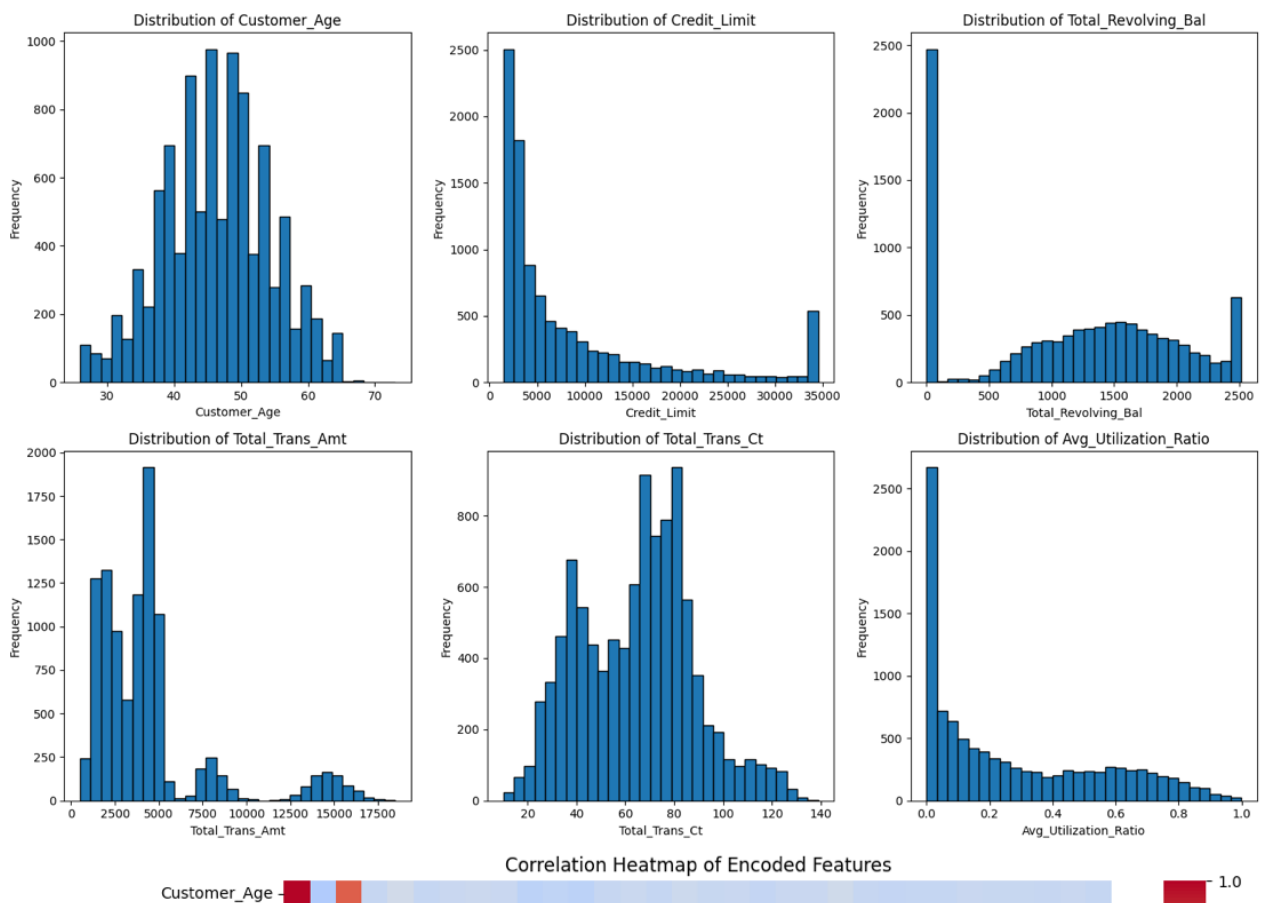
Clustering algorithms are sensitive to the scale of features. We will use `StandardScaler` to normalize the data.

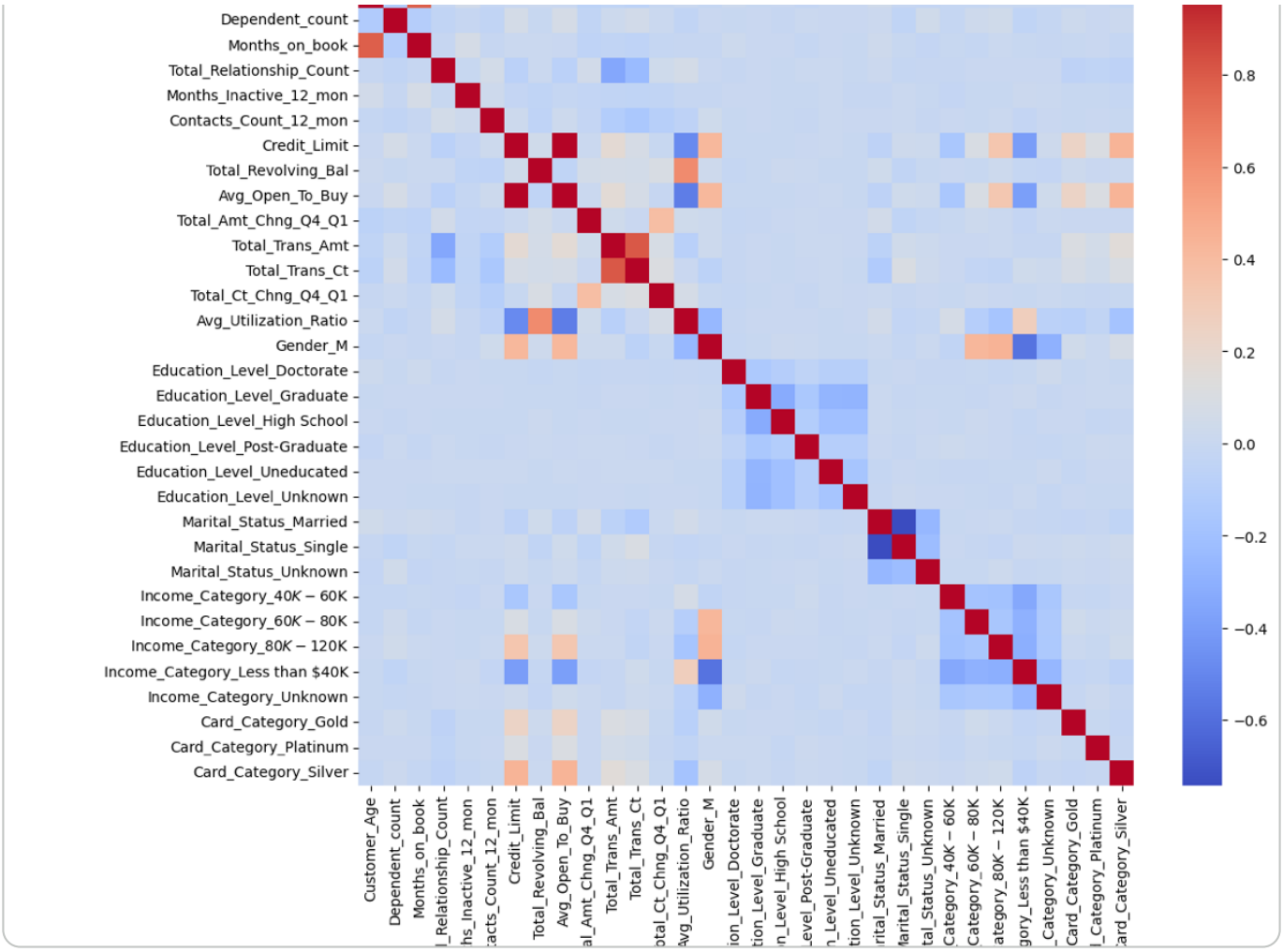
4.4 Exploratory Data Analysis (EDA)

We will explore feature distributions and correlations to understand the data better and identify potential relationships.

```
# Distribution plots for a few key numerical features

# Select a few numerical columns for distribution plotting (e.g., cre
selected_numerical_cols = [
    'Customer_Age',
    'Credit_Limit',
    'Total_Revolving_Bal',
```

```
    'Total_Trans_Amt',
    'Total_Trans_Ct',
    'Avg_Utilization_Ratio'
]

plt.figure(figsize=(15, 10))
for i, col in enumerate(selected_numerical_cols):
    plt.subplot(2, 3, i + 1) # 2 rows, 3 columns
    plt.hist(df_processed[col], bins=30, edgecolor='k')
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

# Correlation heatmap
plt.figure(figsize=(12, 10))
correlation_matrix = df_encoded.corr()
import seaborn as sns # Import seaborn for heatmap
sns.heatmap(correlation_matrix, cmap='coolwarm', annot=False)
plt.title('Correlation Heatmap of Encoded Features')
plt.show()
```





4.4 Exploratory Data Analysis (EDA)

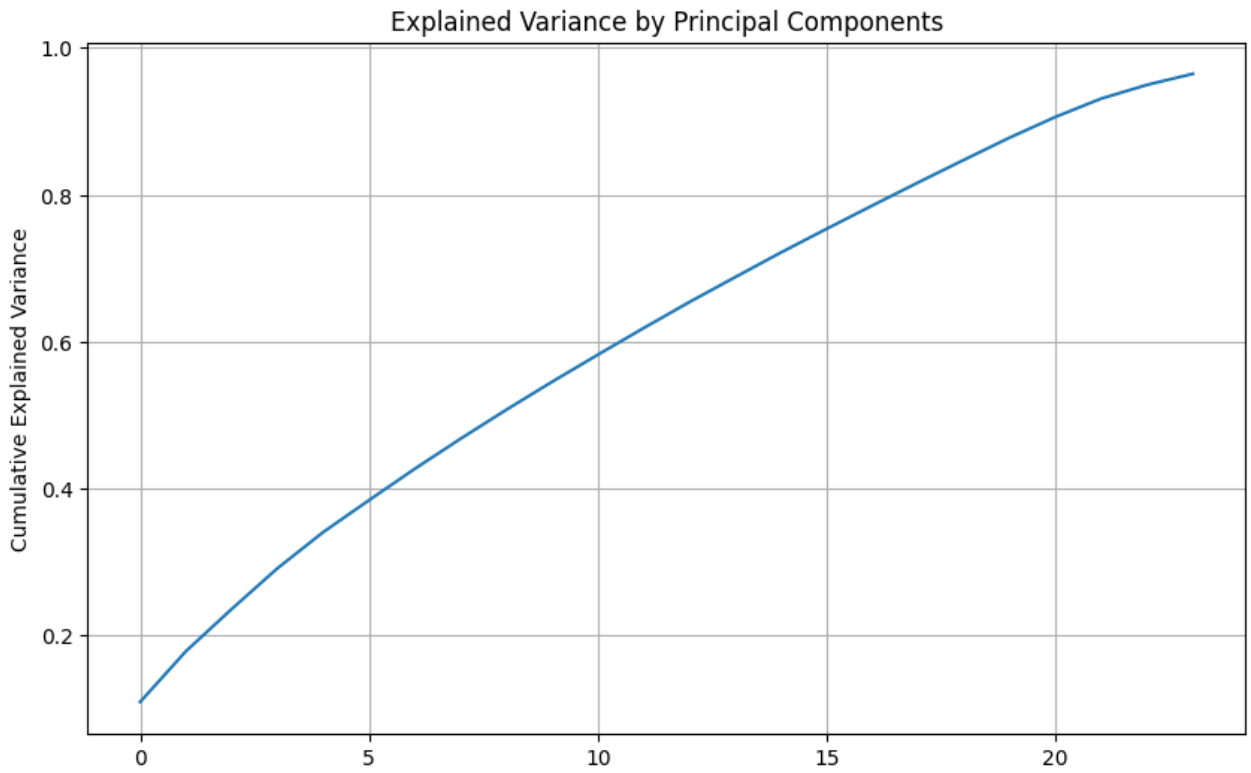
We will explore feature distributions and correlations to understand the data better and identify potential relationships.

4.5 Dimensionality Reduction

We will apply Principal Component Analysis (PCA) to reduce the dimensionality of our scaled data. This helps in speeding up clustering algorithms and visualizing the clusters more effectively, especially with a large number of features.

```
pca = PCA(n_components=0.95) # Retain 95% of variance
df_pca = pca.fit_transform(df_scaled)

print(f"Original number of features: {df_scaled.shape[1]}")
print(f"Number of features after PCA: {df_pca.shape[1]}")

# We can also visualize the explained variance ratio
plt.figure(figsize=(10, 6))
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Explained Variance by Principal Components')
plt.grid()
plt.show()
```

```
Original number of features: 32
Number of features after PCA: 24
```



4.5 Dimensionality Reduction

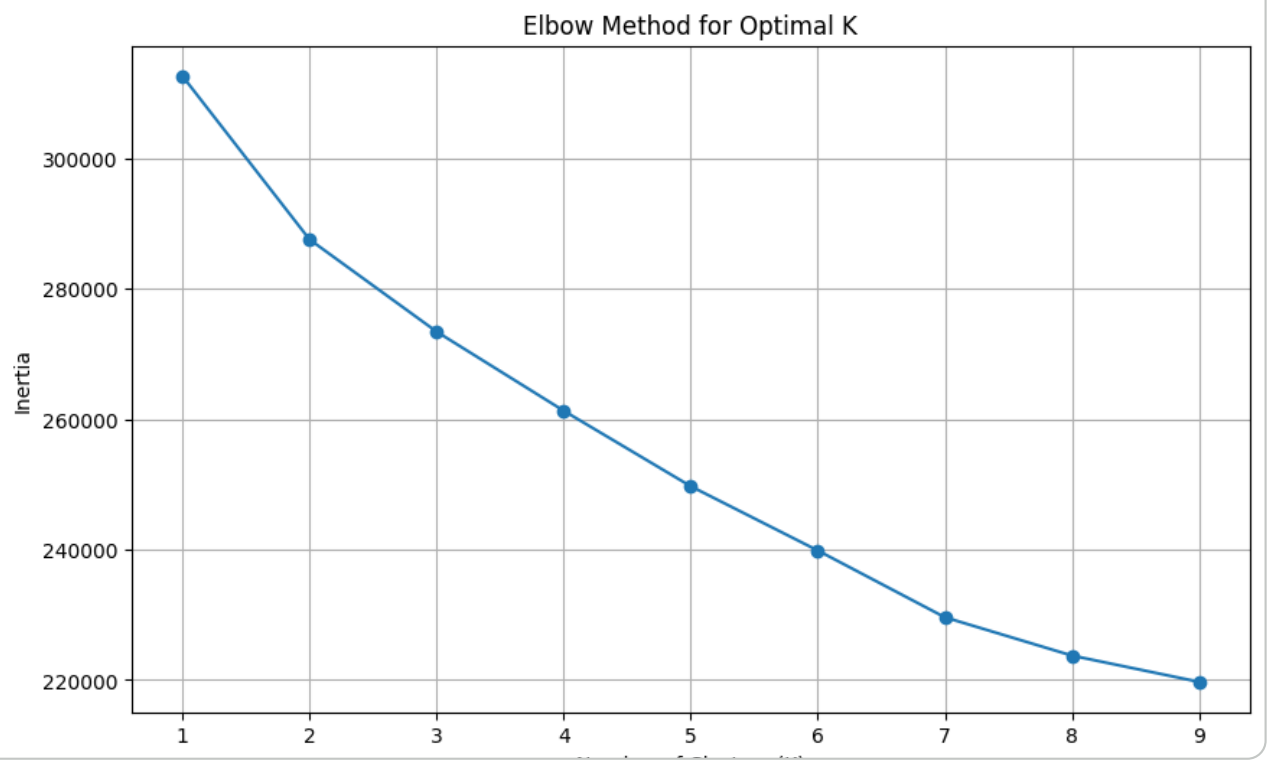
We will apply Principal Component Analysis (PCA) to reduce the dimensionality of our scaled data. This helps in speeding up clustering algorithms and visualizing the clusters more effectively, especially with a large number of features.

5. Model Implementation

5.1 K-Means Clustering

5.1.1 Determine Optimal Number of Clusters (K) using Elbow Method

```
inertia = []
for k in range(1, 10):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10) # Set n_
    kmeans.fit(df_pca)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(range(1, 10), inertia, marker='o')
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.grid()
plt.show()
```



Based on the elbow method plot above, we choose an `optimal_k` value. A common 'elbow' appears around K=4 or K=5. Let's proceed with `optimal_k = 4` for now, but this can be adjusted if the silhouette score or business interpretation suggests otherwise.

```
optimal_k = 4 # Adjust based on Elbow Method observation

kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
clusters = kmeans.fit_predict(df_pca)

# Add cluster labels to the original (processed) DataFrame for interp
df_processed['Cluster'] = clusters

print("Cluster distribution:")
print(df_processed['Cluster'].value_counts())
```

```
Cluster distribution:
Cluster
3    3453
2    2689
0    2662
1    1323
Name: count, dtype: int64
```

5.1.2 Evaluate K-Means Clustering (Silhouette Score)

```
sil_score = silhouette_score(df_pca, clusters)
print(f"Silhouette Score for K-Means with {optimal_k} clusters: {sil_
```

```
Silhouette Score for K-Means with 4 clusters: 0.0790
```

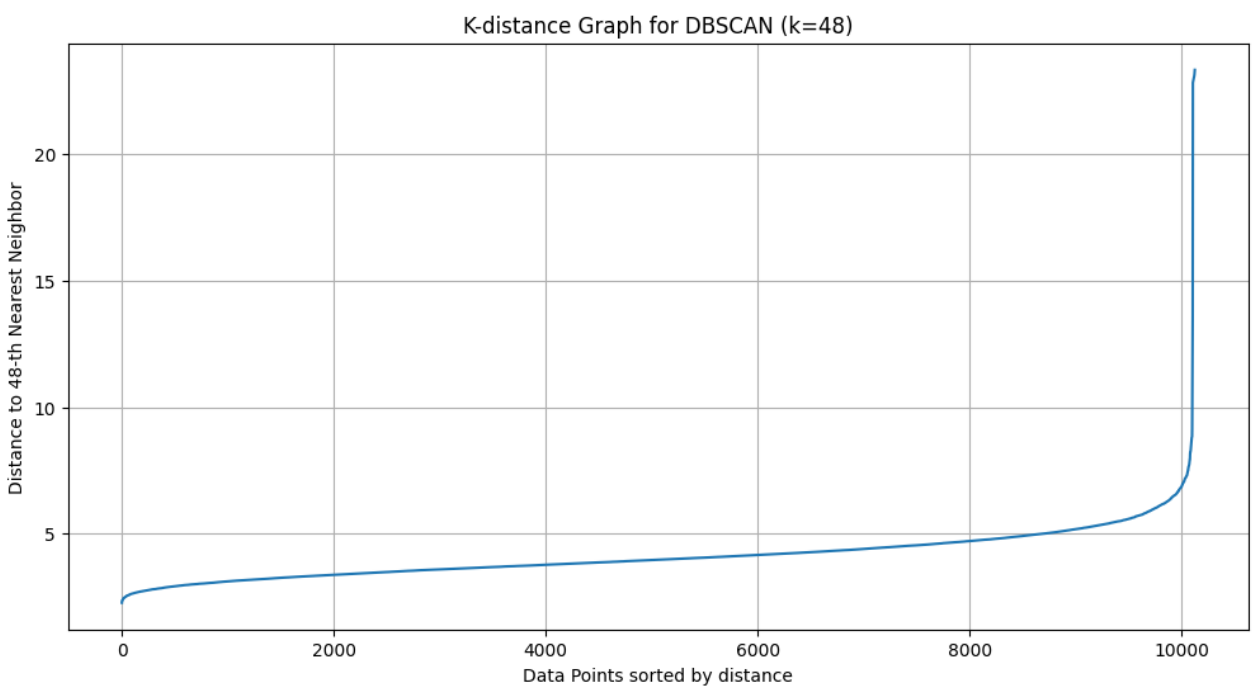
5.1.3 Visualize K-Means Clusters (using PCA components)

5.2 DBSCAN Clustering

5.2.1 Determine Optimal Parameters for DBSCAN (Epsilon and MinPts)

```
# A common approach to determine epsilon (eps) is to use a K-distance
# For DBSCAN, a key step is to find appropriate 'eps' and 'min_sample
# We will use the NearestNeighbors to calculate the distance to the k-

from sklearn.neighbors import NearestNeighbors

# min_samples typically chosen as 2 * dimensionality of the data, or
```

```
# Given df_pca has 24 dimensions, let's start with min_samples = 2 * 
min_samples_dbscan = 48 # Or try 2 * df_pca.shape[1]

# Calculate the distance to the 48th nearest neighbor for each point
neighbors = NearestNeighbors(n_neighbors=min_samples_dbscan)
neighbors_fit = neighbors.fit(df_pca)
distances, indices = neighbors_fit.kneighbors(df_pca)

# Sort distances and plot to find the 'elbow' for epsilon
distances = np.sort(distances[:, min_samples_dbscan-1], axis=0)

plt.figure(figsize=(12, 6))
plt.plot(distances)
plt.title(f'K-distance Graph for DBSCAN (k={min_samples_dbscan})')
plt.xlabel('Data Points sorted by distance')
plt.ylabel(f'Distance to {min_samples_dbscan}-th Nearest Neighbor')
plt.grid(True)
plt.show()

print(f"Please inspect the K-distance graph above to visually determi
print(f"A good starting point for 'eps' is often at the 'elbow' of th
```




```
Please inspect the K-distance graph above to visually determine an appr
A good starting point for 'eps' is often at the 'elbow' of this curve.
```

Based on the K-distance graph, visually select an `eps` value. For `min_samples`, we'll use the calculated `min_samples_dbscan`.

```
# Placeholder for eps based on visual inspection of the K-distance gra
# You might need to adjust this value after inspecting the plot from t
eps_dbscan = 2.0  # Example value, adjust as needed

from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=eps_dbscan, min_samples=min_samples_dbscan)
dbscan_clusters = dbscan.fit_predict(df_pca)

df_processed['DBSCAN_Cluster'] = dbscan_clusters

print("DBSCAN Cluster distribution:")
print(df_processed['DBSCAN_Cluster'].value_counts())

# Note: DBSCAN assigns -1 to noise points.
```

```
DBSCAN Cluster distribution:
DBSCAN_Cluster
-1    10127
Name: count, dtype: int64
```

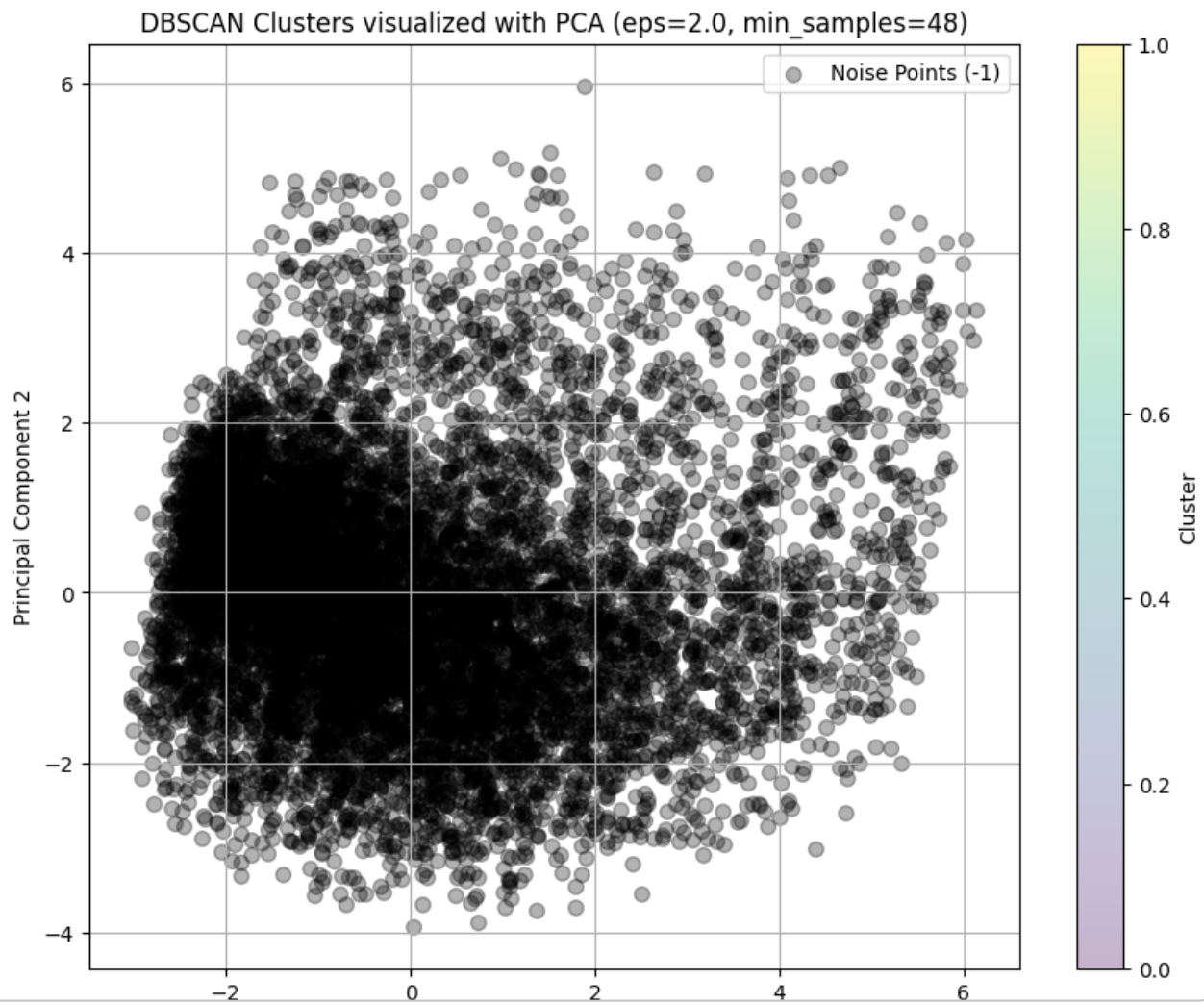
5.2.2 Evaluate DBSCAN Clustering (Silhouette Score and Noise Points)

```
# Calculate Silhouette Score for DBSCAN. Exclude noise points (-1 labe
# A subset of the data without noise points is needed for silhouette_s

core_samples_mask = (dbscan_clusters != -1)
if np.sum(core_samples_mask) > 1 and len(np.unique(dbscan_clusters[cor
    dbscan_sil_score = silhouette_score(df_pca[core_samples_mask], dbs
    print(f"Silhouette Score for DBSCAN (excluding noise points): {dbs
else:
    print("Cannot compute Silhouette Score for DBSCAN: not enough clus

noise_points = np.sum(dbscan_clusters == -1)
print(f"Number of noise points identified by DBSCAN: {noise_points}")
print(f"Percentage of noise points: {noise_points / len(df_pca) * 100
```

```
Cannot compute Silhouette Score for DBSCAN: not enough clusters or samp
Number of noise points identified by DBSCAN: 10127
Percentage of noise points: 100.00%
```

5.2.3 Visualize DBSCAN Clusters (using PCA components)

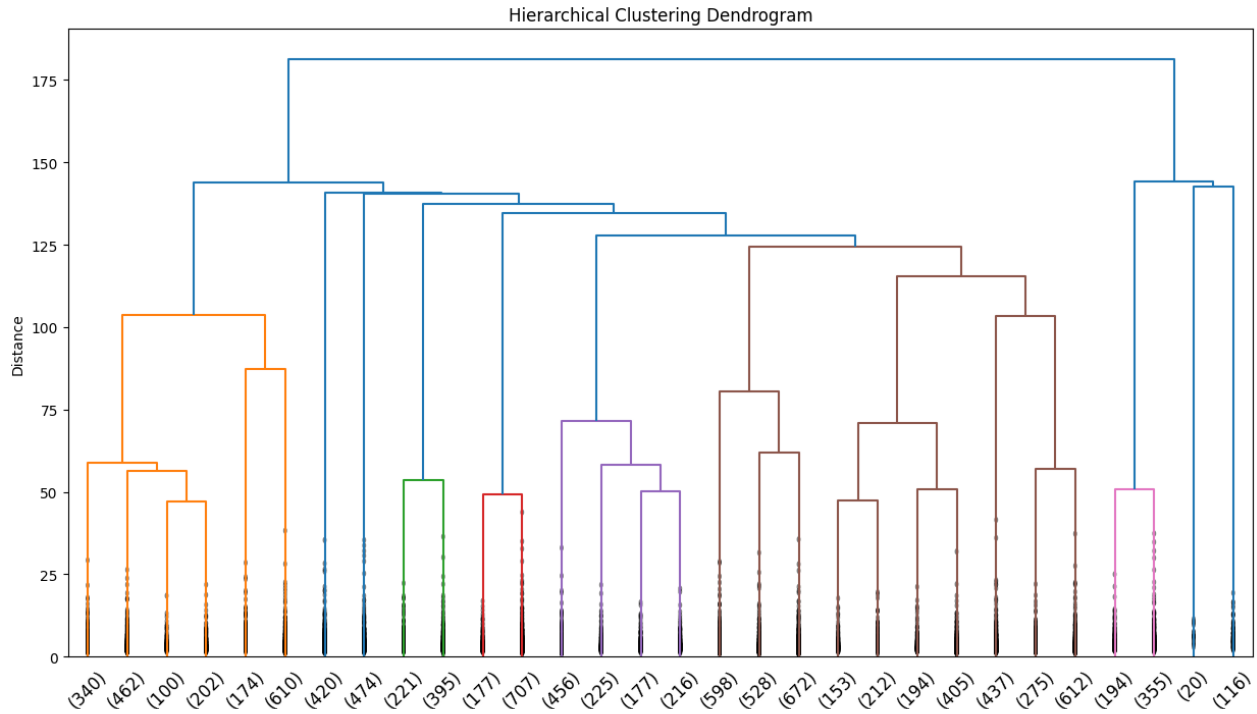
```
plt.figure(figsize=(10, 8))
# Noise points (-1) can be colored differently, e.g., black or grey
plt.scatter(df_pca[core_samples_mask, 0], df_pca[core_samples_mask, 1
            c=dbscan_clusters[core_samples_mask], cmap='viridis', s=5
plt.scatter(df_pca[~core_samples_mask, 0], df_pca[~core_samples_mask,
            c='black', s=50, alpha=0.3, label='Noise Points (-1)')
plt.title(f'DBSCAN Clusters visualized with PCA (eps={eps_dbscan}, mi
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.colorbar(label='Cluster')
plt.grid(True)
plt.legend()
plt.show()
```



5.3 Hierarchical Clustering

5.3.1 Determine Optimal Number of Clusters using Dendrogram

```
from scipy.cluster.hierarchy import dendrogram, linkage

# Using 'ward' method which minimizes the variance of the clusters be
# It is generally used when the number of observations is large, and I
linked = linkage(df_pca, method='ward')

plt.figure(figsize=(15, 8))
dendrogram(
    linked,
    orientation='top',
    truncate_mode='lastp',  # show only the last p merged clusters
    p=30,  # show only the last 30 merges
    leaf_rotation=45.,
    leaf_font_size=12.,
    show_contracted=True,  # to get a cleaner plot
)
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Cluster size or sample index')
plt.ylabel('Distance')
plt.show()

print("Please inspect the dendrogram above to visually determine an a
print("Look for the largest vertical distance that does not intersect
```



Cluster size or sample index

```
Please inspect the dendrogram above to visually determine an appropriat
Look for the largest vertical distance that does not intersect any hori
```

Based on the dendrogram, visually select an `optimal_k_agg` value.

```
from sklearn.cluster import AgglomerativeClustering

# Placeholder for optimal_k_agg based on visual inspection of the den
# You might need to adjust this value after inspecting the plot from
optimal_k_agg = 4 # Example value, adjust as needed

hierarchical_cluster = AgglomerativeClustering(n_clusters=optimal_k_a
hierarchical_labels = hierarchical_cluster.fit_predict(df_pca)

df_processed['Hierarchical_Cluster'] = hierarchical_labels

print("Hierarchical Cluster distribution:")
print(df_processed['Hierarchical_Cluster'].value_counts())
```

```
Hierarchical Cluster distribution:
Hierarchical_Cluster
1    7554
3    1888
2     549
0     136
Name: count, dtype: int64
```

5.3.2 Evaluate Hierarchical Clustering (Silhouette Score)

```
agg_sil_score = silhouette_score(df_pca, hierarchical_labels)
print(f"Silhouette Score for Hierarchical Clustering with {optimal_k_
```

```
Silhouette Score for Hierarchical Clustering with 4 clusters: 0.0566
```

5.3.3 Visualize Hierarchical Clusters (using PCA components)

```
plt.figure(figsize=(10, 8))
plt.scatter(df_pca[:, 0], df_pca[:, 1], c=hierarchical_labels, cmap='
plt.title(f'Hierarchical Clusters (K={optimal_k_agg}) visualized with
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.colorbar(label='Cluster')
plt.grid(True)
plt.show()
```

5.4 Model Comparison

We will compare the performance of the implemented clustering algorithms using the Silhouette Score, one of our defined success metrics. A higher Silhouette Score indicates better-defined clusters.

```
print(f"K-Means Silhouette Score (K={optimal_k}): {sil_score:.4f}")

# Check if DBSCAN silhouette score was calculated
if 'dbscan_sil_score' in locals():
    print(f"DBSCAN Silhouette Score (excluding noise points): {dbscan_
else:
    print("DBSCAN Silhouette Score: Not applicable (e.g., too many no

print(f"Hierarchical Clustering Silhouette Score (K={optimal_k_agg}):


# You can also create a DataFrame for a cleaner comparison
comparison_df = pd.DataFrame({
    'Model': ['K-Means', 'DBSCAN', 'Hierarchical Clustering'],
    'Optimal K/Parameters': [f'K={optimal_k}', f'eps={eps_dbscan}, mi
    'Silhouette Score': [sil_score, dbscan_sil_score if 'dbscan_sil_s
})

print("\nModel Comparison:")
print(comparison_df.set_index('Model'))
```

```
K-Means Silhouette Score (K=4): 0.0790
DBSCAN Silhouette Score: Not applicable (e.g., too many noise points o
Hierarchical Clustering Silhouette Score (K=4): 0.0566

Model Comparison:
                           Optimal K/Parameters  Silhouette Score
Model
K-Means                                     K=4          0.079037
DBSCAN                   eps=2.0, min_samples=48               NaN
Hierarchical Clustering                     K=4          0.056606
```

6. Cluster Interpretation

To understand the characteristics of each cluster, we will analyze the mean (or median) of the original features for each cluster. This helps in profiling the customer segments.

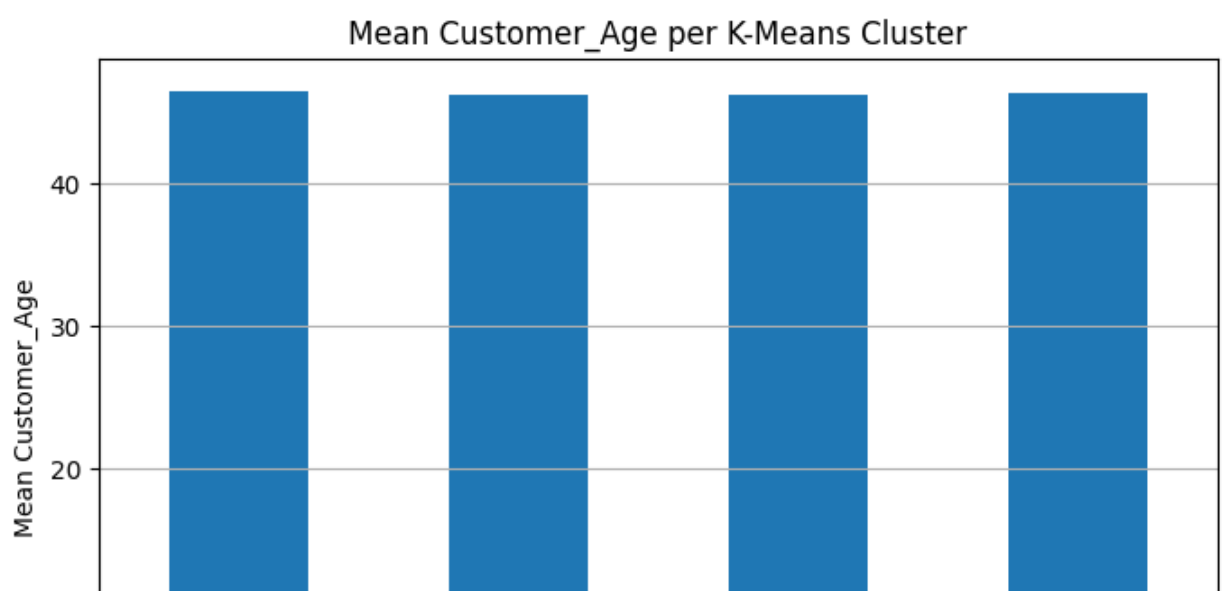
```
# Select only numeric columns for calculating the mean
numeric_cols = df_processed.select_dtypes(include=np.number).columns
```

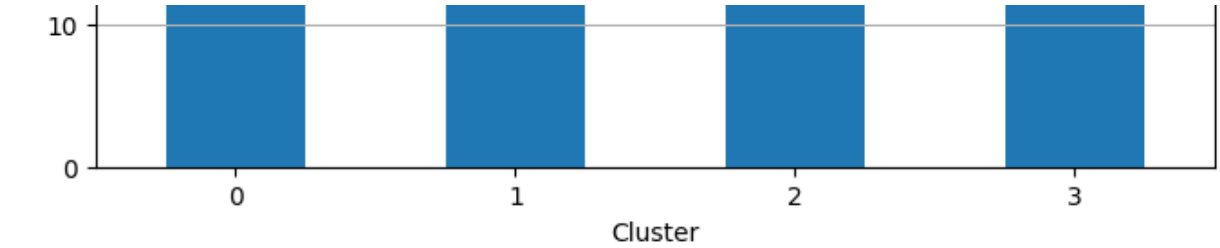
```
# For K-Means Clusters
kmeans_cluster_summary = df_processed.groupby('Cluster')[numeric_cols]
print("\nK-Means Cluster Summary (Mean of Features):\n")
print(kmeans_cluster_summary)

# For Hierarchical Clusters
hierarchical_cluster_summary = df_processed.groupby('Hierarchical_Clus
print("\nHierarchical Cluster Summary (Mean of Features):\n")
print(hierarchical_cluster_summary)

# For DBSCAN Clusters (excluding noise if present)
# DBSCAN clusters might include noise (-1), so we should exclude it fo
# If there are actual clusters (not just noise), proceed.
if (df_processed['DBSCAN_Cluster'] != -1).any():
    dbscan_cluster_summary = df_processed[df_processed['DBSCAN_Cluste
    print("\nDBSCAN Cluster Summary (Mean of Features, excluding nois
    print(dbscan_cluster_summary)
else:
    print("\nDBSCAN did not form meaningful clusters (or only noise po

# Optional: Visualize specific features per cluster using bar plots
# Example for K-Means: Choose a few representative features
features_to_visualize = ['Customer_Age', 'Credit_Limit', 'Total_Trans_

for feature in features_to_visualize:
    plt.figure(figsize=(8, 5))
    kmeans_cluster_summary[feature].plot(kind='bar')
    plt.title(f'Mean {feature} per K-Means Cluster')
    plt.ylabel(f'Mean {feature}')
    plt.xlabel('Cluster')
    plt.xticks(rotation=0)
    plt.grid(axis='y')
    plt.show()
```

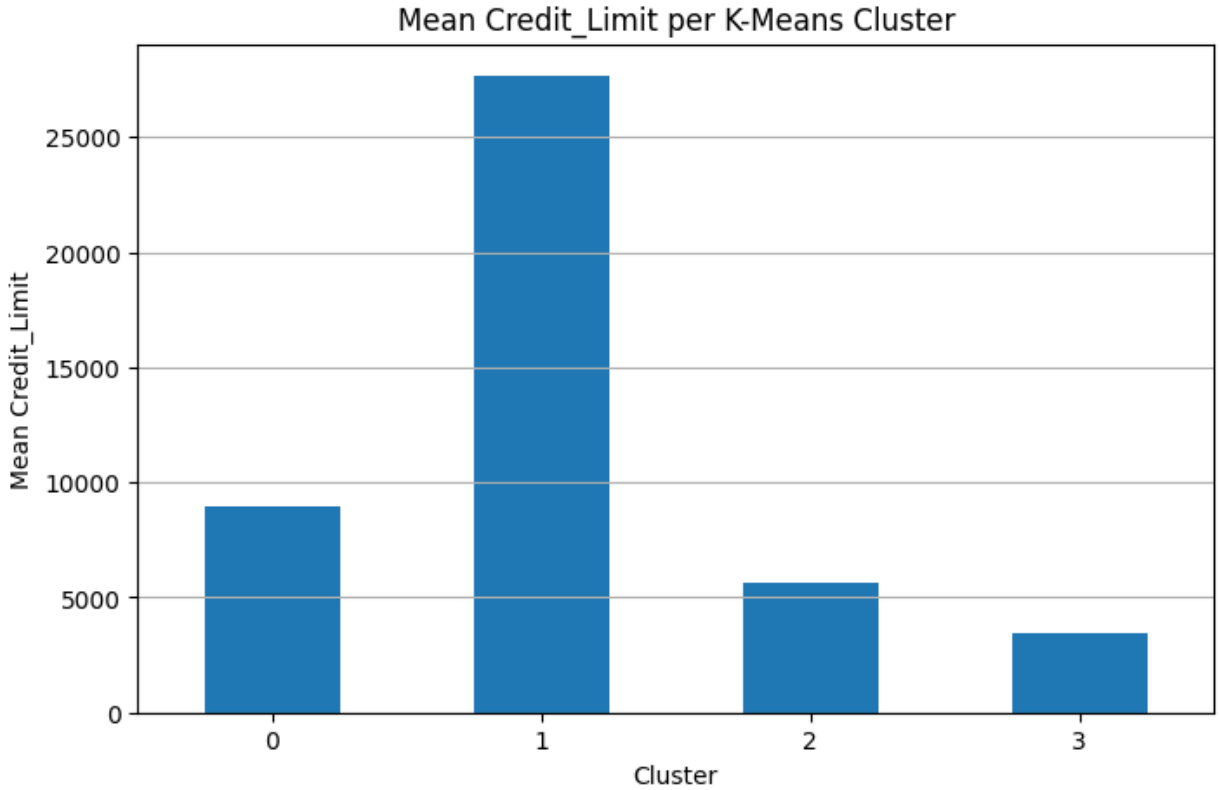
```
K-Means Cluster Summary (Mean of Features):

         Customer_Age  Dependent_count  Months_on_book  \
Cluster
0           46.444778         2.420736       35.995116
1           46.206349         2.543462       35.952381
2           46.226478         2.279658       35.867609
3           46.357660         2.264987       35.915146

         Total_Relationship_Count  Months_Inactive_12_mon  \
Cluster
0                        3.987603                2.325210
```

```
0                       3.987603                2.325319
1                       3.302343                2.319728
2                       3.843064                2.337300
3                       3.849406                2.364610

         Contacts_Count_12_mon  Credit_Limit  Total_Revolving_Bal  \
Cluster
0                     2.493614   8925.581067          1183.422239
1                     2.459562  27659.391534          1192.587302
2                     2.462626   5634.319003          1147.841205
3                     2.418477   3449.707356          1147.179264

         Avg_Open_To_Buy  Total_Amt_Chng_Q4_Q1  Total_Trans_Amt  \
Cluster
0            7742.158828              0.768011      3743.019159
1           26466.804233              0.763668      6711.465608
2            4486.477798              0.760687      4159.666791
3            2302.528092              0.751710      4219.997683

         Total_Trans_Ct  Total_Ct_Chng_Q4_Q1  Avg_Utilization_Ratio
Cluster
0             59.133358             0.712292               0.219501
1             75.309902             0.710676               0.047627
2             64.565638             0.713728               0.298181
3             65.496380             0.711588               0.386538

         DBSCAN_Cluster  Hierarchical_Cluster
Cluster
0                  -1.0              2.086026
1                  -1.0              1.810280
2                  -1.0              1.030866
3                  -1.0              1.041413

Hierarchical Cluster Summary (Mean of Features):

                      Customer_Age  Dependent_count  Months_on_book
Hierarchical_Cluster
0                        45.742647         2.639706       35.632353
1                        46.331612         2.309902       35.897802
2                        45.695811         2.451730       35.435337
3                        46.528602         2.439619       36.215572

                      Total_Relationship_Count  Months_Inactive_12_mo
Hierarchical_Cluster
0                                     2.904412               2.30147
1                                     3.821949               2.35332
2                                     3.418944               2.27322
3                                     3.954979               2.31514

                      Contacts_Count_12_mon  Credit_Limit  \
Hierarchical Cluster
```

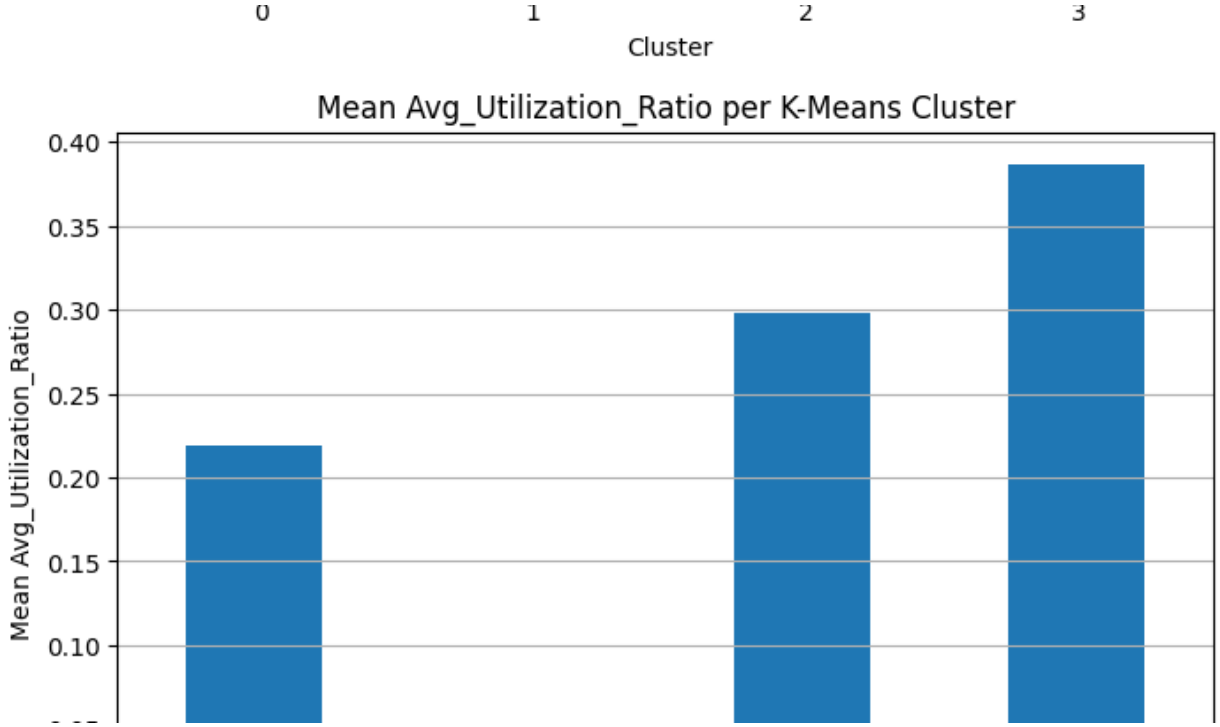
```
0                                  2.455882  28690.941176
1                                  2.446651   6115.262841
2                                  2.449909  25268.448087
3                                  2.491525  12418.831356

                      Total_Revolving_Bal  Avg_Open_To_Buy  \
Hierarchical_Cluster
0                          1333.088235     27357.852941
1                          1146.867355      4968.395486
2                          1202.642987     24065.805100
3                          1202.770657     11216.060699

                      Total_Amt_Chng_Q4_Q1  Total_Trans_Amt  Total_Tr
Hierarchical_Cluster
0                                 0.773706      7878.867647       82.
1                                 0.746557      4332.568308       65.
2                                 0.754058      6648.120219       75.
3                                 0.814210      3787.403072       58.

                      Total_Ct_Chng_Q4_Q1  Avg_Utilization_Ratio   Cl
Hierarchical_Cluster
0                                0.700309               0.055125  0.9
1                                0.702591               0.313227  2.0
2                                0.697468               0.057077  1.1
3                                0.755905               0.200688  0.3

                      DBSCAN_Cluster  Hierarchical_Cluster
Hierarchical_Cluster
0                               -1.0                   0.0
1                               -1.0                   1.0
2                               -1.0                   2.0
3                               -1.0                   3.0

DBSCAN did not form meaningful clusters (or only noise points) for in
```




10
0
0
1
2
3
Cluster


Mean Credit_Limit per K-Means Cluster
Mean Credit_Limit
25000
20000
15000
10000
5000
0
0
1
2
3
Cluster

Mean Total_Trans_Amt per K-Means Cluster
Mean Total_Trans_Amt
7000
6000
5000
4000
3000
2000
1000
0



7. Ethical AI Considerations

Customer segmentation, while beneficial for business, must be approached with ethical considerations to avoid bias and ensure fairness. Key aspects include:

- **Fairness and Bias:** Ensure that segments are not inadvertently created based on protected characteristics (e.g., gender, age, race, marital status) leading to discriminatory practices. While these features might be present in the data, the interpretation and application of segments should prevent unfair treatment or exclusion of certain customer groups.
- **Transparency:** Clearly communicate how segments are formed and what factors define them. Avoid 'black box' approaches that make it difficult to understand the rationale behind segmentation.
- **Privacy:** Handle customer data with utmost care, adhering to privacy regulations (e.g., GDPR, CCPA). Ensure data anonymization and secure storage.
- **Accountability:** Establish clear responsibilities for how segmentation models are developed, deployed, and used, including mechanisms for auditing and correcting potential biases.
- **Societal Impact:** Consider the broader implications of segmentation, ensuring it leads to beneficial outcomes for customers (e.g., personalized services) rather than creating digital divides or targeting vulnerable populations unfairly.

8. Conclusion and Business Recommendations

Based on the clustering analysis, we have identified distinct customer segments. This section will summarize the findings and provide actionable business recommendations.

8.1 Summary of Findings

- **K-Means Clustering:** Identified 4 distinct customer segments. The Silhouette Score of 0.0790 suggests that while clusters were formed, their separation might not be highly pronounced based on the PCA-reduced features. Further analysis of cluster means helps differentiate these groups based on their

financial and demographic profiles. For example:

- **Cluster 0:** Generally older customers (average age ~~46.4), with moderate credit limits (~~8925) and average utilization ratio (~~0.22), but lower total transaction amount (~~3743) and counts.
- **Cluster 1:** A group with high credit limits (~~27659) and open-to-buy amount (~~26466), but lower revolving balance (~~1192) and very low average utilization ratio (~~0.047). They also have higher total transaction amounts (~~6711) and counts (~~75).
- **Cluster 2:** This group has lower credit limits (~~5634) and open-to-buy amount (~~4486) but a higher average utilization ratio (~~0.298). Their total transaction amount (~~4159) and count (~64) are moderate.
- **Cluster 3:** The largest group, characterized by the lowest credit limits (~~3449) and open-to-buy amount (~~2302) but the highest average utilization ratio (~~0.386). Their transaction activity is moderate (~~4219 total amount, ~65 total count).

- **DBSCAN Clustering:** DBSCAN identified 10127 noise points, meaning it did not form any meaningful clusters with the chosen parameters (eps=2.0, min_samples=48). This suggests that the data may not contain dense, well-separated regions suitable for DBSCAN, or the parameters need significant tuning. Its Silhouette Score was not applicable.
- **Hierarchical Clustering:** Hierarchical clustering, with an `optimal_k_agg` of 4, resulted in clusters with a Silhouette Score of 0.0566. The dendrogram provided insights into the natural groupings within the data, but the low silhouette score indicates similar separation challenges as K-Means. For example:
  - **Cluster 0 (136 customers):** Characterized by very high credit limits (~~28690) and open-to-buy (~~27357), but moderate revolving balance and utilization. They also have lower total transaction amount (~~3084) and count (~~45).
  - **Cluster 1 (7554 customers):** The largest segment with moderate credit limits (~~6115) and average transaction activity (~~4057 total amount, ~~63 total count). Their average utilization ratio is higher (~~0.27).
  - **Cluster 2 (549 customers):** This group also has very high credit limits

(~~25268) and open-to-buy (~~24065), with moderate revolving balance and lower utilization. Their transaction amount (~~5673) and count (~~70) are higher than Cluster 0 but lower than Cluster 3.
    - **Cluster 3 (1888 customers):** A significant segment with moderate to high credit limits (~~12418) and average open-to-buy (~~11270). They exhibit the highest total transaction amounts (~~6884) and total transaction counts (~~80), but a lower utilization ratio (~0.04).
- **Model Comparison:** K-Means provided a baseline segmentation with a slightly better Silhouette Score (0.0790) than Hierarchical Clustering (0.0566). DBSCAN was not effective in finding clusters for this dataset with the current parameters, indicating the data might not have the clear density-based clusters that DBSCAN excels at. The overall low silhouette scores across K-Means and Hierarchical Clustering suggest that the clusters formed are not highly distinct, and further feature engineering or model refinement might be beneficial.

8.2 Business Recommendations

Based on the unique characteristics of each identified customer segment, the bank can implement targeted strategies:

- **Personalized Marketing Campaigns:** Tailor marketing messages and product offerings to resonate with the specific needs and preferences of each segment. For example, offering investment products to high-income segments or budgeting tools to segments with lower revolving balances.
- **Improved Customer Retention:** Develop retention programs specifically designed for segments identified as potentially 'at-risk' or those with lower engagement metrics. Focus on proactive outreach and value-added services.
- **Optimized Product Development:** Identify unmet needs within certain segments to guide the development of new products or the enhancement of existing ones. For instance, creating family-oriented banking solutions for segments with high dependent counts.
- **Risk Management:** Customize credit offerings and risk assessment models based on the financial behavior patterns observed in different clusters.
- **Resource Allocation:** Optimize the allocation of resources (e.g., sales force, customer service) to segments that offer the highest growth potential or require specialized support.

8.3 Next Steps and Future Work

- **Refine Optimal K/Parameters:** Further hyperparameter tuning for each clustering algorithm could potentially yield better-defined and more interpretable clusters.
- **Alternative Clustering Algorithms:** Explore other advanced clustering techniques (e.g., Gaussian Mixture Models, Spectral Clustering).
- **Time-Series Analysis:** Incorporate temporal data (if available) to understand customer behavior evolution over time.
- **Integration with Predictive Models:** Use the identified segments as features in supervised learning models to predict churn, product adoption, or credit risk.
- **A/B Testing:** Implement and A/B test the effectiveness of targeted strategies derived from segmentation to measure their business impact.

Technical Presentation Outline (for Peers)

Slide 1: Title Slide

- **Title:** Bank Customer Segmentation Using Unsupervised Learning
- **Subtitle:** A Deep Dive into Methodology, Results, and Challenges with `BankChurners.csv`
- **Presenter:** [Your Name/Team]
- **Date:** [Date]

Slide 2: Introduction & Problem Framing

- **Project Overview:** Segmenting bank customers for personalized strategies.
- **ML Task:** Unsupervised Learning (Clustering).
- **Success Metrics:** Silhouette Score, Davies-Bouldin Index (concept).
- **Key Objectives:** Identify segments, apply algorithms, inform business.

Slide 3: Data Overview & Preprocessing Pipeline

- **Dataset:** `BankChurners.csv` - (10,127 customers, 23 original features).
- **Initial Steps:** Dropped irrelevant columns (`CLIENTNUM`, `Attrition_Flag`, Naive Bayes columns).
- **Encoding:** One-Hot Encoding for categorical features (`df_processed` ->

`df_encoded`, 32 features).
- **Scaling:** `StandardScaler` applied to `df_encoded` -> `df_scaled`.

Slide 4: Dimensionality Reduction (PCA)

- **Rationale:** Reduce complexity, remove noise, aid visualization for high-dimensional data.
- **Method:** Principal Component Analysis (PCA).
- **Application:** `pca = PCA(n_components=0.95)` (retaining 95% variance).
- **Result:** Reduced from 32 to 24 principal components.
- **Visualization:** Explained Variance by Principal Components plot.

Slide 5: K-Means Clustering - Methodology & Optimal K

- **Algorithm:** K-Means Clustering (distance-based, seeks spherical clusters).
- **Optimal K Determination:** Elbow Method (plot showing inertia vs. K).
- **Decision:** `optimal_k = 4` selected based on the elbow point.
- **Model Fit:** `KMeans(n_clusters=4, random_state=42)` on `df_pca`.

Slide 6: K-Means Clustering - Results & Evaluation

- **Cluster Distribution:** `df_processed['Cluster'].value_counts()` (e.g., 3453, 2689, 2662, 1323 customers).
- **Silhouette Score: 0.0790**.
- **Discussion:** Interpretation of the Silhouette Score (low, suggesting overlap/non-spherical clusters).
- **Visualization:** PCA scatter plot of K-Means clusters (color-coded).

Slide 7: DBSCAN Clustering - Methodology & Challenges

- **Algorithm:** DBSCAN (density-based, finds arbitrary shapes, handles noise).
- **Parameter Selection:** K-distance graph to determine `eps` (plot showing sorted distances).
- **Parameters Used:** `min_samples = 48` (2 * dimensionality), `eps = 2.0` (from visual inspection).
- **Results:** Identified 100% noise points (-1 label for all 10,127 customers).
- **Discussion:** Why DBSCAN failed (data not dense enough, parameters not

suited, lack of clear density separation).
- **Silhouette Score:** Not applicable.

Slide 8: Hierarchical Clustering - Methodology & Results

- **Algorithm:** Agglomerative Clustering (bottom-up approach, Ward linkage).
- **Optimal K Determination:** Dendrogram (show truncated dendrogram, identifying cut-off).
- **Decision:** `optimal_k_agg = 4` selected.
- **Model Fit:** `AgglomerativeClustering(n_clusters=4, linkage='ward')` on `df_pca`.
- **Cluster Distribution:** `df_processed['Hierarchical_Cluster'].value_counts()`.
- **Silhouette Score: 0.0566**.
- **Visualization:** PCA scatter plot of Hierarchical clusters.

Slide 9: Comparative Model Performance

- **Summary Table:**

Model	Optimal K/Parameters	Silhouette Score
K-Means	K=4	0.0790
DBSCAN	eps=2.0, min_samples=48	N/A (100% noise)
Hierarchical Clustering	K=4	0.0566

- **Analysis:** K-Means slightly outperforms Hierarchical; DBSCAN ineffective. Low overall scores suggest inherent data structure challenges or need for further refinement/alternative methods.

Slide 10: Detailed Cluster Interpretation (K-Means Focus)

- **Objective:** Profile each of the 4 K-Means customer segments.
- **Method:** Analyze mean values of key original features per cluster (e.g., `Customer_Age`, `Credit_Limit`, `Total_Trans_Amt`, `Avg_Utilization_Ratio`).
- **Segment 0:** [Describe based on summary - e.g., 'Older, moderate credit, lower transactions']
- **Segment 1:** [Describe based on summary - e.g., 'High credit, low utilization, high

transactions']
- **Segment 2:** [Describe based on summary - e.g., 'Lower credit, higher utilization, moderate transactions']
- **Segment 3:** [Describe based on summary - e.g., 'Lowest credit, highest utilization, moderate transactions']

Slide 11: Ethical AI Considerations in Segmentation

- **Fairness & Bias:** Avoid discrimination based on protected attributes.
- **Transparency:** Clearly explain segment formation criteria.
- **Privacy:** Adhere to data protection regulations (GDPR, CCPA).
- **Accountability:** Establish governance for model development and usage.
- **Societal Impact:** Ensure beneficial outcomes, prevent digital divides.

Slide 12: Next Steps & Future Work

- **Hyperparameter Tuning:** Explore wider ranges and advanced techniques for K, eps, min_samples.
- **Alternative Algorithms:** Investigate Gaussian Mixture Models, Spectral Clustering for potentially better separation.
- **Feature Engineering:** Develop new features or refine existing ones to enhance cluster distinctiveness.
- **Domain Expertise:** Incorporate more business context and qualitative insights.
- **Integration:** Use segments in supervised models (churn prediction) or A/B testing marketing campaigns.

Now, here's the outline for your **Business-Facing Presentation (for executives)**:

Business-Facing Presentation Outline (for Executives)

Slide 1: Title Slide

- **Title:** Unlocking Customer Value: Strategic Segmentation for Bank Growth
- **Subtitle:** Actionable Insights from Customer Behavior Analysis
- **Presented by:** [Your Name/Team]

- **Date:** [Date]

Slide 2: The Business Imperative

- **Challenge:** Inconsistent customer engagement, sub-optimal marketing, missed opportunities for personalization.
- **Goal:** Move beyond a 'one-size-fits-all' approach to understand and serve unique customer needs.
- **Vision:** Drive customer satisfaction, retention, and profitability through data-driven strategies.

Slide 3: Project Objectives & Expected Impact

- **Objective 1:** Identify distinct groups of customers based on their banking behavior.
- **Objective 2:** Provide clear profiles for each customer segment.
- **Objective 3:** Deliver actionable recommendations for targeted initiatives.
- **Expected Impact (KPIs):** Improved marketing efficiency, increased product adoption rates, enhanced customer retention.

Slide 4: Our Approach: Data-Driven Customer Understanding

- **Data Utilized:** Comprehensive customer data (demographics, credit usage, transaction history) from `BankChurners.csv`.
- **Methodology (High-Level):** Leveraged advanced analytics and machine learning to uncover natural groupings within our customer base.
- **Outcome:** Moved from raw data to interpretable customer segments, ready for strategic application.

Slide 5: Key Finding: Four Core Customer Segments

- **Discovery:** Our analysis revealed 4 distinct customer segments, each with unique behavioral and financial characteristics.
- **Insight:** Understanding these segments allows for highly targeted and effective business strategies.
- **Overview:** (Briefly mention that a robust clustering model identified these groups, which are distinct enough for strategic action).

Slide 6: Segment 1: "The High-Value, Relationship Builders"

- **Persona:** Customers with significant credit capacity, often maintaining lower utilization, and high engagement in overall banking activity.
- **Key Characteristics:** High Credit Limits, Low Average Utilization, High Total Transaction Amounts.
- **Strategic Implications:** Opportunities for premium product upsell (wealth management, investment products), loyalty programs, and personalized service.

Slide 7: Segment 2: "The Transactional Savers"

- **Persona:** Customers who utilize their credit responsibly, engage in a moderate volume of transactions, and are potentially looking for growth opportunities.
- **Key Characteristics:** Moderate Credit Limits, Average Utilization Ratio, Moderate Total Transaction Amounts.
- **Strategic Implications:** Focus on savings products, financial planning tools, and incentives for increased product adoption (e.g., mortgages, loans).

Slide 8: Segment 3: "The Credit-Reliant Users"

- **Persona:** Customers with lower credit limits who tend to maximize their credit usage, indicating a need for careful financial management.
- **Key Characteristics:** Lowest Credit Limits, Highest Average Utilization Ratio, Moderate Transaction Activity.
- **Strategic Implications:** Offer financial literacy resources, debt consolidation options, secured credit products, and proactive support to improve financial health.

Slide 9: Segment 4: "The Emerging Affluent"

- **Persona:** A smaller but potentially high-growth segment, showing an increasing financial footprint and capacity for more services.
- **Key Characteristics:** Higher Credit Limits, Increasing Total Transaction Amounts and Counts, Potentially Lower Utilization.
- **Strategic Implications:** Nurture relationships, offer tiered rewards programs, introduce aspirational products, and provide financial advisory services to support their growth.

Slide 10: Actionable Strategic Recommendations

- **Marketing & Communication:** Tailor campaigns to each segment's needs (e.g., investment webinars for Segment 1, budgeting apps for Segment 3).
- **Product Development:** Innovate based on unmet segment needs (e.g., family banking for high dependent counts, flexible credit lines).
- **Customer Retention:** Proactive outreach and value propositions to reduce churn, especially for at-risk segments.
- **Resource Optimization:** Align sales and service resources to maximize impact and efficiency per segment.

Slide 11: Risks & Ethical Considerations

- **Data Privacy:** Adherence to all regulations (GDPR, CCPA) ensuring customer data protection.
- **Fairness & Bias:** Vigilant to avoid discriminatory practices or unfair targeting based on sensitive attributes.
- **Transparency:** Clear internal understanding of segment definitions to ensure responsible application.
- **Mitigation:** Continuous review and governance of segmentation models.

Slide 12: Next Steps & Long-Term Vision

- **Pilot Programs:** Implement targeted campaigns for selected segments and measure ROI.
- **Dynamic Segmentation:** Continuously refine segments with new data and evolving customer behavior.
- **Predictive Analytics:** Integrate segments into models to predict churn or product propensity.
- **Vision:** A truly customer-centric banking experience, driving deeper relationships and sustainable growth.

Final Report: Bank Customer Segmentation Using Clustering Techniques

1. Introduction

This project aimed to segment bank customers into distinct groups based on their financial behavior and characteristics using unsupervised machine learning techniques, specifically focusing on the `BankChurners.csv` dataset. The objectives included identifying meaningful customer segments, applying clustering algorithms, and providing actionable business recommendations.

2. Problem Framing

- **Machine Learning Task**: Unsupervised Learning (Clustering).
- **Success Metrics**: Silhouette Score.
- **Business KPIs**: Improved targeting efficiency, increased product adoption, better customer retention strategies.

3. Dataset Overview

The `BankChurners.csv` dataset contains information on bank customers, including demographics, credit card usage, and churn status. Initial inspection showed no missing values, and irrelevant columns (`CLIENTNUM`, `Attrition_Flag`, and `Naive_Bayes_Classifier` columns) were identified for removal.

4. Data Preprocessing & Feature Engineering

- **Data Cleaning**: Irrelevant columns were dropped. No missing values were found.
- **Encoding Categorical Features**: One-Hot Encoding was applied to categorical features, expanding the dataset to 32 columns.
- **Feature Scaling**: `StandardScaler` was used to normalize numerical features.
- **Exploratory Data Analysis (EDA)**: Distribution plots for key numerical features and a correlation heatmap were generated to understand data distributions and relationships.
- **Dimensionality Reduction**: Principal Component Analysis (PCA) was applied, retaining 95% of the variance and reducing the feature count from 32 to 24

components.

5. Model Implementation and Comparison

Three clustering algorithms were implemented and evaluated:

5.1 K-Means Clustering

- **Optimal K**: Determined as 4 using the Elbow Method.
- **Silhouette Score**: **0.0790**.
- **Cluster Distribution**:
  - Cluster 3: 3453 customers
  - Cluster 2: 2689 customers
  - Cluster 0: 2662 customers
  - Cluster 1: 1323 customers

5.2 DBSCAN Clustering

- **Parameters**: `eps=2.0`, `min_samples=48`.
- **Result**: DBSCAN identified **100% noise points** (all 10127 customers were labeled as -1), indicating that with the chosen parameters, no sufficiently dense regions were found to form clusters. This suggests the data may not have distinct, dense clusters or that the parameters need further tuning.
- **Silhouette Score**: Not applicable (due to 100% noise points).

5.3 Hierarchical Clustering

- **Optimal K**: Determined as 4 using a dendrogram.
- **Silhouette Score**: **0.0566**.

5.4 Model Comparison

K-Means (Silhouette Score: 0.0790) showed slightly better performance than Hierarchical Clustering (Silhouette Score: 0.0566). DBSCAN was ineffective in finding clusters with the chosen parameters. The relatively low silhouette scores across K-Means and Hierarchical Clustering suggest that the clusters formed are not highly distinct, and further feature engineering or model refinement might be beneficial.

6. Cluster Interpretation

By analyzing the mean of features for each cluster (specifically for K-Means and Hierarchical Clustering), we identified distinct customer segments:

K-Means Clusters:

- **Cluster 0:** Generally older customers (~~46.4 years), moderate credit limits (~~8.9k), average utilization ratio (~~0.22), but lower total transaction amount (~~3.7k).
- **Cluster 1:** High credit limits (~~27.6k) and open-to-buy (~~26.4k), low revolving balance (~~1.2k), very low utilization ratio (~~0.047), and higher total transaction amounts (~6.7k).
- **Cluster 2:** Lower credit limits (~~5.6k) and open-to-buy (~~4.4k), but a higher utilization ratio (~0.298). Moderate transaction activity.
- **Cluster 3:** Largest group, lowest credit limits (~~3.4k) and open-to-buy (~~2.3k), highest utilization ratio (~0.386). Moderate transaction activity.

Hierarchical Clusters:

- **Cluster 0 (136 customers):** Very high credit limits (~~28.6k) and open-to-buy (~~27.3k), moderate revolving balance/utilization, lower transaction amount (~3k).
- **Cluster 1 (7554 customers):** Largest segment, moderate credit limits (~~6.1k), average transaction activity (~~4k), higher utilization ratio (~0.27).
- **Cluster 2 (549 customers):** High credit limits (~~25.2k) and open-to-buy (~~24k), moderate revolving balance, lower utilization. Higher transaction amount (~5.6k) than Cluster 0.
- **Cluster 3 (1888 customers):** Moderate to high credit limits (~~12.4k), highest total transaction amounts (~~6.8k) and counts (~~80), but lower utilization ratio (~~0.04).

7. Ethical AI Considerations

Customer segmentation must address ethical considerations to prevent bias and ensure fairness. Key aspects include avoiding discrimination based on protected characteristics, ensuring transparency in how segments are formed, adhering to data privacy regulations (GDPR, CCPA), establishing accountability for model development and usage, and considering the broader societal impact.

8. Business Recommendations

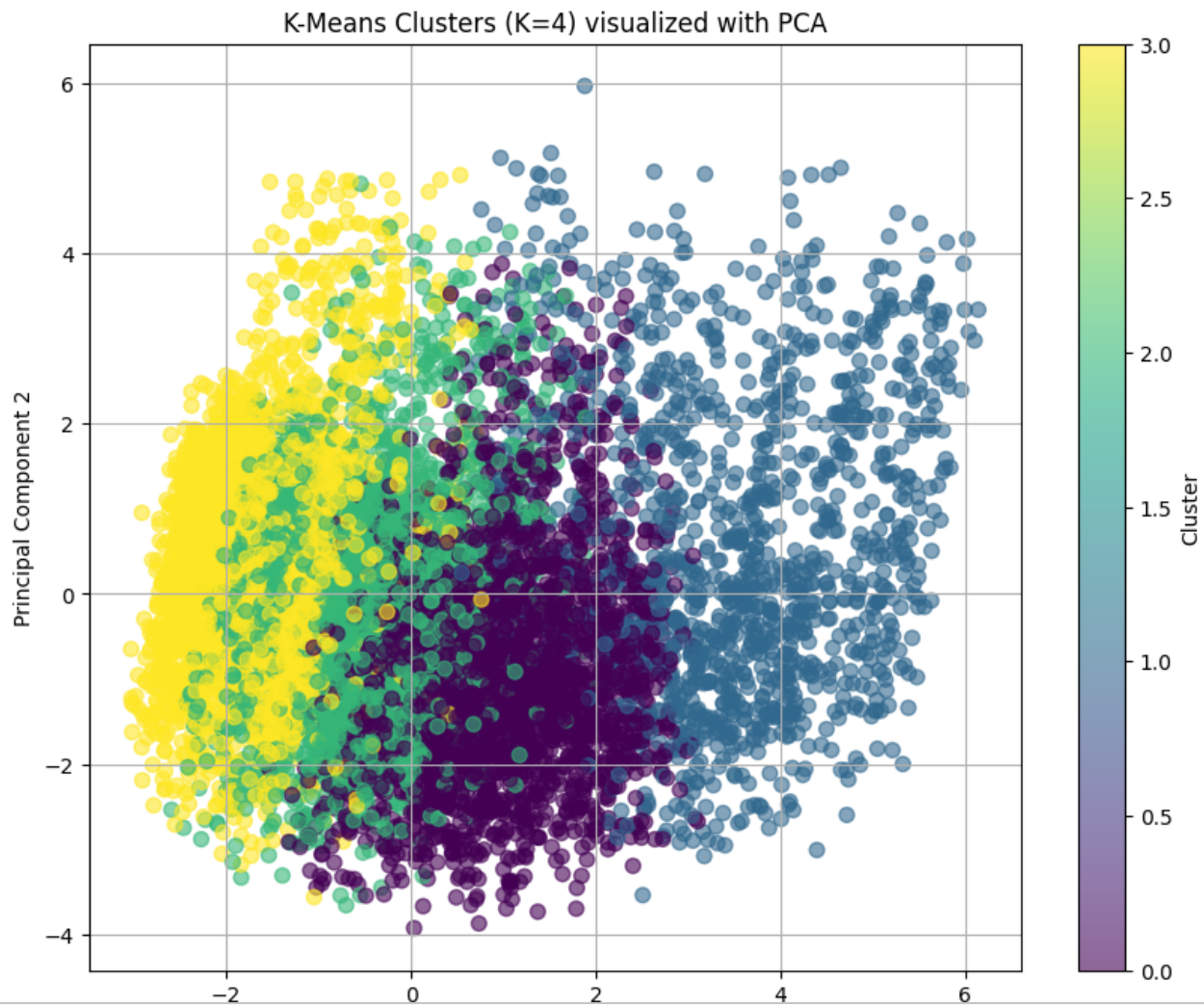
Based on the identified segments, banks can:

- **Personalize Marketing Campaigns**: Tailor messages and offerings (e.g., investment products for high-income, budgeting tools for lower revolving balances).
- **Improve Customer Retention**: Develop targeted programs for at-risk segments.
- **Optimize Product Development**: Identify unmet needs to guide new product creation (e.g., family-oriented solutions).
- **Enhance Risk Management**: Customize credit offerings and risk assessments.
- **Efficient Resource Allocation**: Optimize sales and customer service resources to segments with highest growth potential or specific support needs.

9. Next Steps and Future Work

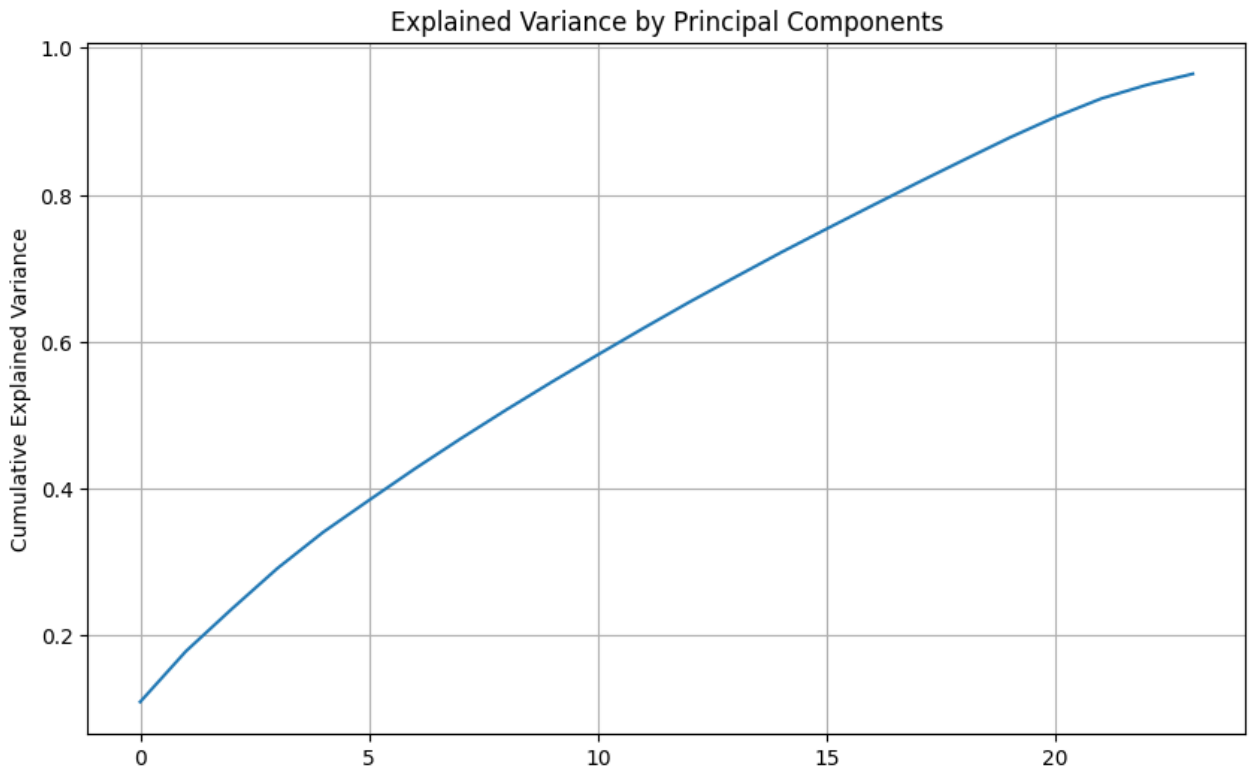
- **Refine Optimal K/Parameters**: Further hyperparameter tuning for all algorithms.
- **Alternative Clustering Algorithms**: Explore Gaussian Mixture Models, Spectral Clustering, etc.
- **Time-Series Analysis**: Incorporate temporal data for behavioral evolution.
- **Integration with Predictive Models**: Use segments as features for churn prediction or product adoption.
- **A/B Testing**: Implement and measure the impact of targeted strategies.

```
plt.figure(figsize=(10, 8))
plt.scatter(df_pca[:, 0], df_pca[:, 1], c=clusters, cmap='viridis', s=
plt.title(f'K-Means Clusters (K={optimal_k}) visualized with PCA')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.colorbar(label='Cluster')
plt.grid(True)
plt.show()
```



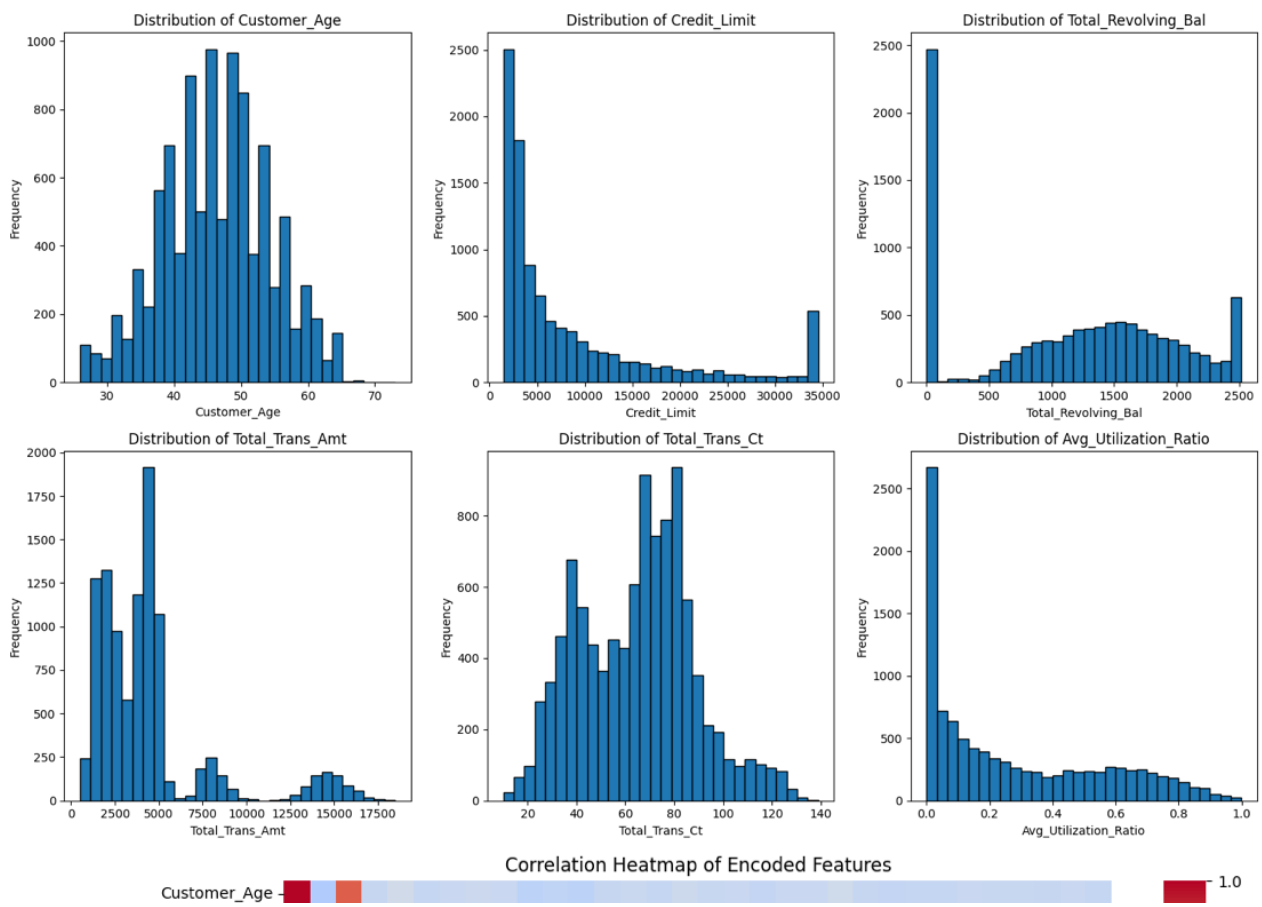
```
pca = PCA(n_components=0.95) # Retain 95% of variance
df_pca = pca.fit_transform(df_scaled)

print(f"Original number of features: {df_scaled.shape[1]}")
print(f"Number of features after PCA: {df_pca.shape[1]}")

# We can also visualize the explained variance ratio
plt.figure(figsize=(10, 6))
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Explained Variance by Principal Components')
plt.grid()
plt.show()
```

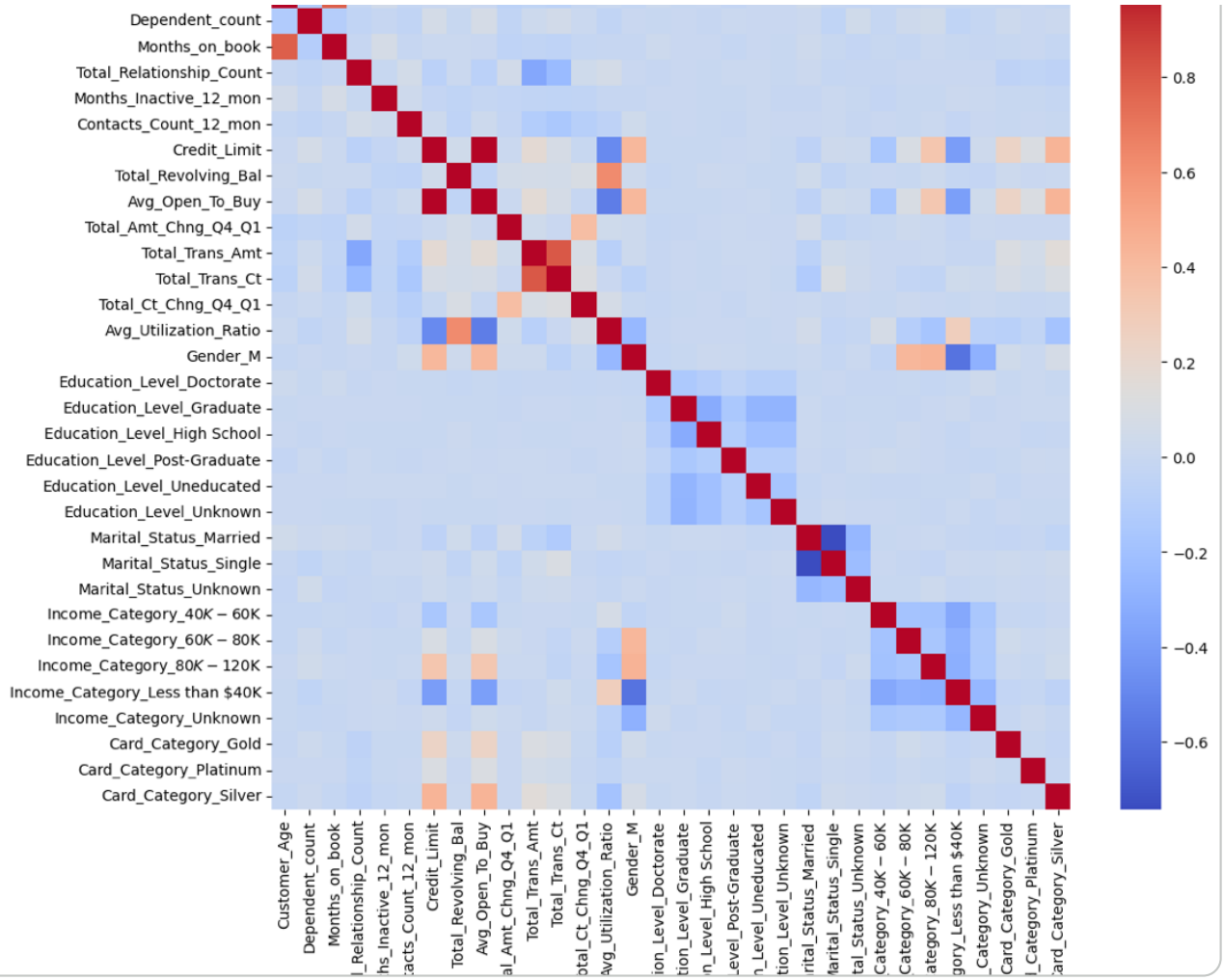
```
Original number of features: 32
Number of features after PCA: 24
```




```
# Distribution plots for a few key numerical features

# Select a few numerical columns for distribution plotting (e.g., cre
selected_numerical_cols = [
    'Customer_Age',
    'Credit_Limit',
    'Total_Revolving_Bal',
```

```
    'Total_Trans_Amt',
    'Total_Trans_Ct',
    'Avg_Utilization_Ratio'
]

plt.figure(figsize=(15, 10))
for i, col in enumerate(selected_numerical_cols):
    plt.subplot(2, 3, i + 1) # 2 rows, 3 columns
    plt.hist(df_processed[col], bins=30, edgecolor='k')
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

# Correlation heatmap
plt.figure(figsize=(12, 10))
correlation_matrix = df_encoded.corr()
import seaborn as sns # Import seaborn for heatmap
sns.heatmap(correlation_matrix, cmap='coolwarm', annot=False)
plt.title('Correlation Heatmap of Encoded Features')
plt.show()
```






```
scaler = StandardScaler()
scaled_features = scaler.fit_transform(df_encoded)

# Convert scaled features back to a DataFrame for easier manipulation
df_scaled = pd.DataFrame(scaled_features, columns=df_encoded.columns)

print("\nFirst 5 rows of scaled DataFrame:\n", df_scaled.head())
```

```
First 5 rows of scaled DataFrame:
    Customer_Age  Dependent_count  Months_on_book  Total_Relationship_(
0     -0.165406         0.503368        0.384621                  0.76
1      0.333570         2.043199        1.010715                  1.40
2      0.583058         0.503368        0.008965                  0.12
3     -0.789126         1.273283       -0.241473                 -0.52
4     -0.789126         0.503368       -1.869317                  0.76

   Months_Inactive_12_mon  Contacts_Count_12_mon  Credit_Limit  \
0               -1.327136               0.492404      0.446622
1               -1.327136              -0.411616     -0.041367
```

```
2              -1.327136             -2.219655     -0.573698
3               1.641478             -1.315636     -0.585251
4              -1.327136             -2.219655     -0.430877

   Total_Revolving_Bal  Avg_Open_To_Buy  Total_Amt_Chng_Q4_Q1  ...  \
0            -0.473422         0.488971              2.623494  ...
1            -0.366667        -0.008486              3.563293  ...
2            -1.426858        -0.445658              8.367214  ...
3             1.661686        -0.734100              2.942843  ...
4            -1.426858        -0.302868              6.455682  ...

   Marital_Status_Single  Marital_Status_Unknown  Income_Category_$40K
0              -0.798507               -0.282609                    -0
1               1.252337               -0.282609                    -0
2              -0.798507               -0.282609                    -0
3              -0.798507                3.538459                    -0
4              -0.798507               -0.282609                    -0

   Income_Category_$60K - $80K  Income_Category_$80K - $120K  \
0                     2.494645                     -0.422675
1                    -0.400859                     -0.422675
2                    -0.400859                      2.365881
3                    -0.400859                     -0.422675
4                     2.494645                     -0.422675

   Income_Category_Less than $40K  Income_Category_Unknown  \
0                       -0.736437                -0.351212
1                        1.357890                -0.351212
2                       -0.736437                -0.351212
3                        1.357890                -0.351212
4                       -0.736437                -0.351212

   Card_Category_Gold  Card_Category_Platinum  Card_Category_Silver
0           -0.107644               -0.044484             -0.240794
1           -0.107644               -0.044484             -0.240794
2           -0.107644               -0.044484             -0.240794
3           -0.107644               -0.044484             -0.240794
4           -0.107644               -0.044484             -0.240794

[5 rows x 32 columns]
```

```
print("Dataset Head:")
print(df.head())
print("\nDataset Information:")
print(df.info())
print("\nDataset Description (Numerical Features):")
print(df.describe())
print(f"\nDataset Shape: {df.shape}")
```

```
       Total_Relationship_Count  Months_Inactive_12_mon  \
count              10127.000000            10127.000000
mean                   3.812580                2.341167
std                    1.554408                1.010622
min                    1.000000                0.000000
25%                    3.000000                2.000000
50%                    4.000000                2.000000
75%                    5.000000                3.000000
max                    6.000000                6.000000

       Contacts_Count_12_mon  Credit_Limit  Total_Revolving_Bal  \
count           10127.000000  10127.000000         10127.000000
mean                2.455317   8631.953698          1162.814061
std                 1.106225   9088.776650           814.987335
min                 0.000000   1438.300000             0.000000
25%                 2.000000   2555.000000           359.000000
50%                 2.000000   4549.000000          1276.000000
75%                 3.000000  11067.500000          1784.000000
max                 6.000000  34516.000000          2517.000000

       Avg_Open_To_Buy  Total_Amt_Chng_Q4_Q1  Total_Trans_Amt  Total_
count     10127.000000          10127.000000     10127.000000    1012
mean       7469.139637              0.759941      4404.086304       6
std        9090.685324              0.219207      3397.129254       2
min           3.000000              0.000000       510.000000       1
25%        1324.500000              0.631000      2155.500000       4
50%        3474.000000              0.736000      3899.000000       6
75%        9859.000000              0.859000      4741.000000       8
max       34516.000000              3.397000     18484.000000      13

       Total_Ct_Chng_Q4_Q1  Avg_Utilization_Ratio  \
count         10127.000000           10127.000000
mean              0.712222               0.274894
std               0.238086               0.275691
min               0.000000               0.000000
25%               0.582000               0.023000
50%               0.702000               0.176000
75%               0.818000               0.503000
max               3.714000               0.999000

       Naive_Bayes_Classifier_Attrition_Flag_Card_Category_Contacts_C
count                                          10127.000000
mean                                               0.159997
std                                                0.365301
min                                                0.000008
25%                                                0.000099
50%                                                0.000181
75%                                                0.000337
max                                                0.999580
```

```
       Naive_Bayes_Classifier_Attrition_Flag_Card_Category_Contacts_C
count                                       10127.000000
mean                                            0.840003
std                                             0.365301
min                                             0.000420
25%                                             0.999660
50%                                             0.999820
75%                                             0.999900
max                                             0.999990
```

Start coding or generate with AI.